Jour 1 : Nettoyage du texte

Preprocessing, normalisation et preparation des donnees textuelles

Introduction au Natural Language Processing

Le **Natural Language Processing (NLP)** est une branche de l'intelligence artificielle qui se concentre sur le traitement et l'analyse du langage naturel par les ordinateurs.

Taches NLP principales

- **Classification texte** : spam vs ham
- **Analyse sentiments** : positif / negatif
- **NER** : reconnaissance d'entites
- **Traduction** : machine translation
- **Summarization** : resume automatique

Applications metier

- Chatbots et assistants virtuels
- Analyse de retours clients
- Detection de fraude textuelle
- Moteurs de recherche
- Recommandations personnalisees

> **NLP vs Machine Learning classique :** Le texte est non-structure et hautement dimensionnel. Nous devons transformer le texte brut en representations numeriques comprenables par les modeles ML.

? Pourquoi preparer le texte ?

Le texte brut contient du bruit, des inconsistances et des caracteres inutiles qui reduisent les performances des modeles ML.

Bruit

- Ponctuation excessive
- Caracteres speciaux
- Nombres inutiles
- HTML / symboles

Variabilite

- Casses differentes (A vs a)
- Espaces extras
- Accents / diacritiques
- Formes differentes du meme mot

Redondance

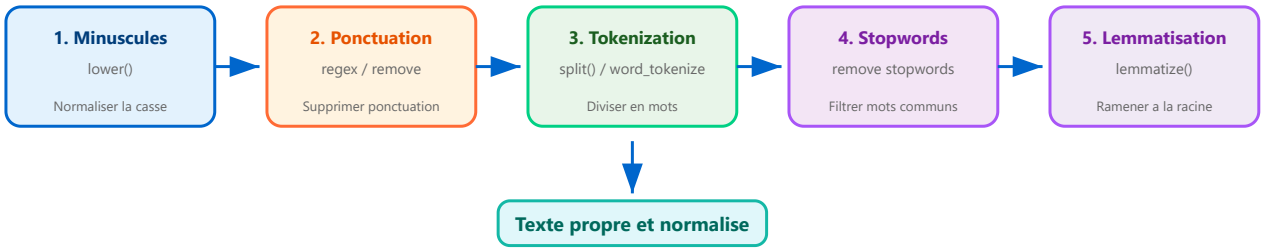
- Stopwords communs (le, la, de)
- Mots peu informatifs
- Mots vides inutiles

- Repetitions

Resultat : Un texte propre et normalise produit de meilleurs features pour les modeles ML et reduit la dimensionnalite de 20 a 40%.

Pipeline de nettoyage du texte

Le preprocessing suit typiquement ces etapes dans cet ordre :



> **Note :** Cet ordre n'est pas absolu. Il varie selon le contexte, la langue et l'objectif de l'analyse. Quelques projets peuvent ne pas avoir besoin de toutes les etapes.

Normalisation : casse et espaces

Les premieres transformations eliminent les inconsistances de casse et d'espacement.

PYTHON

```python
# Normalisation basique
import re

texte = "  Bonjour le MONDE!   Comment allez-vous?  "

# Convertir en minuscules
texte_lower = texte.lower()
print(texte_lower)
# Output: "  bonjour le monde!   comment allez-vous?  "

# Supprimer les espaces extras (debut, fin, multiples)
texte_clean = re.sub(r'\s+', ' ', texte_lower).strip()
print(texte_clean)
# Output: "bonjour le monde! comment allez-vous?"

# Supprimer accents (diacritiques)
import unicodedata

def remove_accents(text):
    nfkd = unicodedata.normalize('NFKD', text)
    return ''.join([c for c in nfkd if not unicodedata.combining(c)])

texte_no_accent = remove_accents("élève, élèves, œuvre")
print(texte_no_accent)
# Output: "eleve, eleves, oeuvre"
```

> ⚠ **Attention :** La suppression des accents peut perdre des informations importantes dans certaines langues. C'est un choix de design dependant du contexte.

⌨ Suppression de la ponctuation

La ponctuation et les caracteres speciaux sont rarement utiles pour l'analyse textuelle et doivent generalement etre supprimes.

PYTHON

```python
# Approche 1 : avec regex
import re

texte = "Hello, world! How are you? #NLP @python123"

# Supprimer tout sauf lettres, chiffres et espaces
texte_clean = re.sub(r'[^a-zA-Z0-9\s]', '', texte)
print(texte_clean)
# Output: "Hello world How are you NLP python123"

# Approche 2 : avec string module
import string

texte = "Hello, world!"
translator = str.maketrans('', '', string.punctuation)
texte_clean = texte.translate(translator)
print(texte_clean)
# Output: "Hello world"

# Approche 3 : regex avec preservation des tirets
texte = "c'est l'exemple, non? oui!"
texte_clean = re.sub(r'[^\w\s-]', '', texte)
print(texte_clean)
# Output: "cest lexemple non oui"
```

> **Conseil :** Parfois garder la ponctuation peut etre utile (par ex. "Mr." vs "Mr" dans la detection de noms).

✋ Suppression des stopwords

Les **stopwords** sont des mots tres communs avec peu de valeur semantique : articles, prepositions, conjonctions, etc.

Stopwords francais courants

le, la, de, et, un, une, des, que, qui, ou, pour, par, avec, sur, dans, sans, a, au, dont, ce, se, plus, mais, ne, pas, mon, ton, son, nos, vos, ses, je, tu, il, nous, vous, ils, mon, ta, sa, ma, notre, votre

Utilisation avec NLTK

PYTHON

```python
from nltk.corpus import stopwords

# Charger stopwords francais
stop_words = set(stopwords.words('french'))

texte = "ceci est un exemple de texte"
mots = texte.split()
filtered = [m for m in mots if m not in stop_words]
print(filtered)
# ['exemple', 'texte']
```

⚠ **Attention :** Supprimer les stopwords ne convient pas toujours. En sentiment analysis, "ne", "pas" sont cruciaux pour "n'est pas bon" vs "est bon".

Stemming : reduction a la racine

Le **stemming** reduit les mots a leur **radical** (racine) en supprimant les suffixes. C'est une approche brutale mais rapide.

PYTHON

```python
from nltk.stem.snowball import SnowballStemmer

# Creation du stemmer pour le francais
stemmer = SnowballStemmer('french')

# Exemples de stemming
mots = ['courant', 'courir', 'courra', 'courses', 'couriers']

for mot in mots:
    radical = stemmer.stem(mot)
    print(f"{mot:15} -> {radical}")

# Output:
# courant         -> courant
# courir          -> cour
# courra          -> courr
# courses         -> cours
# couriers        -> courier
```

> **Algorithmiquement :** Stemming utilise des regles basees sur des patterns pour couper les suffixes. Tres rapide mais moins precis que la lemmatisation.

Lemmatisation : reduction semantique

La **lemmatisation** reduit les mots a leur **lemme** (forme canonique) en utilisant la grammaire et les dictionnaires. Plus precis que le stemming.

PYTHON

```python
import spacy

# Charger le modele francais de SpaCy
nlp = spacy.load('fr_core_news_sm')

# Exemples de lemmatisation
texte = "Les chevaux courent. Ils couraient vite."
doc = nlp(texte)

for token in doc:
    print(f"{token.text:15} -> {token.lemma_:15} ({token.pos_})")

# Output:
# Les             -> le              (DET)
# chevaux         -> cheval          (NOUN)
# courent         -> courir          (VERB)
# Ils             -> il              (PRON)
# couraient       -> courir          (VERB)
# vite            -> vite            (ADV)
```

Avantage : SpaCy utilise un modele syntaxique pour comprendre la grammaire. "courir", "courant", "couru" -> tous au lemme "courir".

⚖️ Stemming vs Lemmatisation

Deux approches complementaires avec leurs avantages et inconvenients respectifs.

Critere	Stemming	Lemmatisation
Approche	Regles syntaxiques simples	Dictionnaire + grammaire
Vitesse	Tres rapide	Plus lent (tagging syntaxique)
Precision	Moins precise, erreurs possibles	Plus precise et semantiquement correcte
Resultat	Radical (peut ne pas etre un mot reel)	Lemme (mot reel, forme canonique)
Exemple	"courir", "courra" -> "cour"	"courir", "courra" -> "courir"
Ressources requises	Peu (liste de regles)	Important (dictionnaires, modeles)

ℹ Quand utiliser quoi ?

- **Stemming** : Quand la vitesse est cruciale, les datasets tres gros, classification de base
- **Lemmatisation** : Quand la precision semantique est importante, NER, relation extraction

Pipeline complet de preprocessing

Voici une fonction complete qui combine tous les nettoyages etapes :

PYTHON

```python
import re
import unicodedata
import spacy
from nltk.corpus import stopwords

def preprocess_texte(texte, remove_stopwords=True, lemmatize=True):
    """Preprocessing complet d'un texte en francais."""
    # 1. Minuscules et espaces
    texte = texte.lower()
    texte = re.sub(r'\s+', ' ', texte).strip()

    # 2. Supprimer ponctuation et caracteres speciaux
    texte = re.sub(r'[^a-z0-9\s]', '', texte)

    # 3. Tokenization et stopwords
    tokens = texte.split()
    if remove_stopwords:
        stop_words = set(stopwords.words('french'))
        tokens = [t for t in tokens if t not in stop_words]

    # 4. Lemmatisation (optionnel)
    if lemmatize:
        nlp = spacy.load('fr_core_news_sm')
        doc = nlp(' '.join(tokens))
        tokens = [token.lemma_ for token in doc]

    return ' '.join(tokens)

# Exemple d'utilisation
texte = "Bonjour! Comment allez-vous aujourd'hui?"
resultat = preprocess_texte(texte)
```

```
print(resultat)
# Output: "bonjour comment allezvous"
```

Cas limites et bonnes pratiques

Cas problematiques

- **Noms propres** : peut etre important de conserver
- **Contractions** : "n'est" vs "nest"
- **Nombres** : eliminer ou garder ?
- **URLs et emails** : traitement special
- **Emojis** : contexte-dependant

Bonnes pratiques

- Adapter le preprocessing au contexte et tache
- Garder des donnees originales pour comparaison
- Documenter votre pipeline de preprocessing
- Valider que le preprocessing améliore les resultats
- Être coherent entre train et test

Exercice

Creez un preprocessing pour des avis de films avec emojis, mentions d'utilisateurs (@user), et URLs. Quel est le meilleur ordre d'etapes ?

PYTHON

```python
# Solution : preprocessing adapte aux avis avec metadata
import re

def preprocess_reviews(texte):
```

```
    # 1. Supprimer URLs
    texte = re.sub(r'http\S+|www\S+', '', texte)

    # 2. Supprimer mentions (@user)
    texte = re.sub(r'@\w+', '', texte)

    # 3. Transformer emojis en texte (optionnel)
    # texte = emoji.demojize(texte)

    # 4. Minuscules et espaces
    texte = texte.lower()
    texte = re.sub(r'\s+', ' ', texte).strip()

    # 5. Ponctuation
    texte = re.sub(r'[^a-z0-9\s]', '', texte)

    return texte
```